



WinCE5.0 eBoot and Kernel Porting Guide

Version 0.6

Shmt Co., Ltd.



REVISION HISTORY PAGE

Issue	Rev No.	Originator	Details of Change	Date
1	0.1	DJKIM	Initial Version	2006-09-22
2	0.2	DJKIM	Function & Variable name change	2006-09-26
3	0.3	DJKIM	SkyWalker register mapping	2006-09-29
4	0.4	DJKIM	PLATFORM directory modify	2006-09-29
5	0.5	DJKIM	SkyWalker50 BSP directory modify	2006-10-10
6	0.6	DJKIM	Bootloader & Kernel & Serial & Ethernet driver modify and added	2006-10-23

Confidential



Technical documents for WinCE5.0 eBoot and Kernel porting

Copyright © 2006 Shmt Limited. All rights reserved.

Confidentiality Status

This document is **NOT OPEN ACCESS**. The distribution is restricted to certified company.

This document is **CONFIDENTIAL**.

Feedback on this document

If you have any comments on this document, please contact your channel to our company.

Shmt Web address

<http://www.shmt.com>

Confidential



Contents

1. INTRODUCTION	8
2. OverView	9
3. Boot Loader	12
3.1. Boot Loader Startup Sequence	12
4. Kernel	15
4.1. Kernel Startup Sequence	15
5. HowTo Porting	19
5.1. Boot Loader Development	19
5.2. Kernel Development	20
5.3. PLATFORM\SkyWalker50\ directory	23
5.3.1. Directory Overview	23
5.3.2. Sub-directory and Files	23
5.4. PLATFORM\COMMON\ directory	29
5.4.1. Directory Overview	29
5.4.2. Sub-directory and Files	30
5.5. PUBLIC\COMMON\ directory	31
5.5.1. Directory Overview	31
5.5.2. Sub-directory and Files	31
5.6. Functional and Source code porting	33
5.6.1. PLATFORM\SkyWalker50\ directory	33
6. Source Analysis	35
6.1. Cache Function	35
6.2. MMU Function	35
6.3. Data Abort Function	36
6.4. eBoot Loader	36
6.4.1. Src\Bootloader\Eboot\ directory	36
6.4.2. Src\Bootloader\stepldr\ directory	38
6.5. Kernel	39
6.5.1. Src\kernel\oal\ directory	39
6.6. Header Files	40
6.6.1. Src\Inc\ directory	40
6.7. Startup Code	41
6.7.1. Src\Bootloader\Eboot\startup.s	41
6.7.2. Src\Bootloader\Stepldr\startup.s	42
6.7.3. Src\Kernel\Oal\startup.s	43
7. Debugging and Testing	44
7.1. Boot Loader	47
7.2. Kernel	47
APPENDIX	49
A. Reference (MSDN Site)	49
B. Important Functions and Files	49
C. OAL Layer Functions	49



D. OAL Global Variables	55
-------------------------------	----

Confidential

Figures

[Fig - 1] The Whole structur of the WinCE	9
[Fig - 2] BSP Develop Process.....	9
[Fig - 3] PLATFORM\SkyWalker50\ directory	23
[Fig - 4] Src\Cesysgen\files directory	24
[Fig - 5] Src\Files directory	24
[Fig - 6] Src\Bootloader\ directory	25
[Fig - 7] Src\Common\ directory.....	26
[Fig - 8] Src\Drivers\ directory	26
[Fig - 9] Src\Inc\ directory	28
[Fig - 10] Src\Kernel\ directory	29
[Fig - 11] SRC\ARM\SMT\SMT926A\ directory	30
[Fig - 12] OAK\CSP\ARM\SMT\SMT926A\ directory	31
[Fig - 13] Bootloader startup code flow chart.....	42
[Fig - 14] Step-stone loader startup code flow chart	43
[Fig - 15] Kernel startup code flow chart.....	44
[Fig - 16] Platform Builder Execution Window	46
[Fig - 17] Remote Kernel Tracker Window.....	46
[Fig - 18] CeLog file Load Figure.....	46
[Fig - 19] Remote Tools in the Platform Builder.....	47



Tables

[Table - 1] Boot Loader Startup Sequence	12
[Table - 2] Return codes for the OEMPreDownload	14
[Table - 3] Kernel Startup Sequence.....	16
[Table - 4] Debugging Resource Overview.....	45
[Table - 5] Remote Tools in the Platform Builder.....	48

Confidential



1. INTRODUCTION

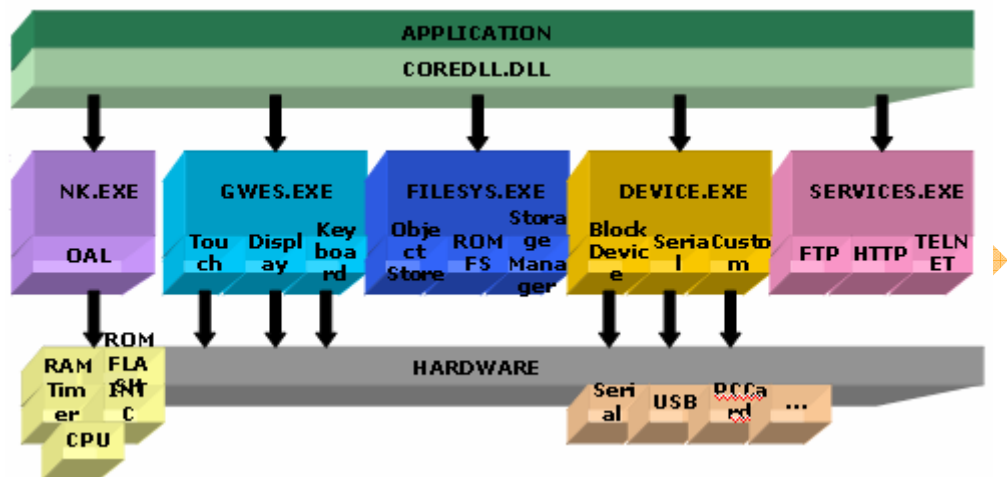
This document treats the porting of boot loader and kernel in the WinCE5.0.

BSP(Board Support Package) referred to S3C2440(Samsung arm9 process) of the WinCE4.2.

Confidential

2. OverView

The following figure is the whole structure of the WinCE.

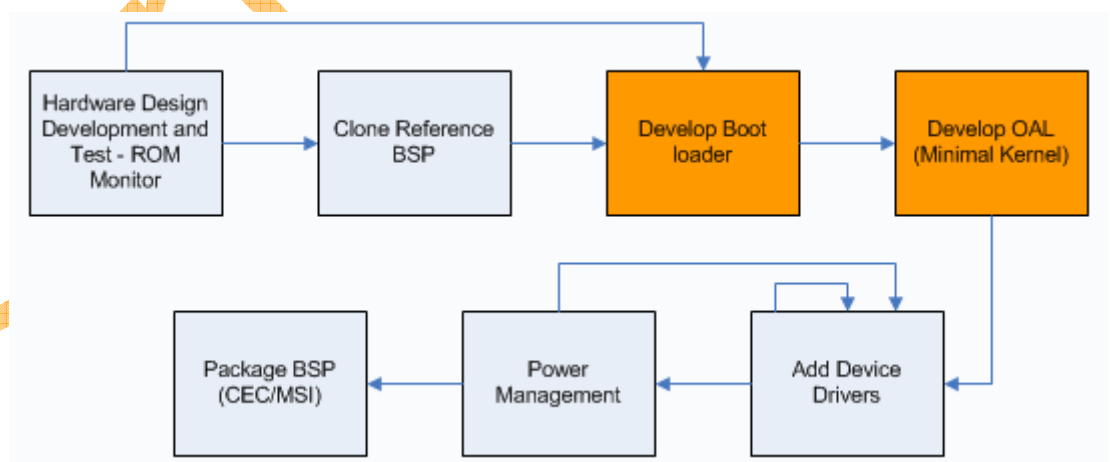


* OAL : OEM Adaptation Layer

* GWES : Graphics Windowing and Events Sub-system

[Fig - 1] The Whole structure of the WinCE

Like many development tasks, the process for developing a BSP involves high-level and low-level tasks. For example, at the highest level, you must select the hardware platform, and at the lowest level, you must develop a boot loader and the OEM abstraction layer (OAL) for the kernel. The following illustration shows the high-level steps needed to develop the low-level software for a BSP.



[Fig - 2] BSP Develop Process



You begin by selecting or developing the hardware platform that the BSP will target. This includes developing a board with a CPU that can run the Windows CE operating system (OS), selecting the peripheral devices that will interface with the device, and including the interfaces required to support BSP development and debugging. For example, you might not need to include support for serial connections on a device for consumers, but a serial connection is typically important for low-level software development. You should consider populating your device with a debug header that can interface to a debug board for development, but can be depopulated before the device is used for production.

After you have identified and created the hardware platform, you must develop a BSP that will run Windows CE on the device. Often, you can modify a BSP from Microsoft or a third party, which will greatly reduce your BSP development work. The process of starting with a BSP from Microsoft is called BSP cloning. You can find information about this process in the Windows CE Help documentation. If a BSP is not available, you will have to develop one entirely on your own. Because developing a BSP entirely on your own is a significant amount of work, Microsoft recommends that you start with a BSP from Microsoft or a third party.

At this point, you move to the low-level development tasks by developing a boot loader that can reside in persistent memory on the device. The boot loader's primary function is to initialize enough of the hardware and CPU to allow the hardware to communicate with the development environment for downloading a Windows CE-based run-time image. You can extend the boot loader's functionality to be used for any other purpose that your device requires. Later sections of this paper discuss the low-level startup functions that you must implement in the boot loader.

After a boot loader has been developed, you download it and write it into persistent memory on the device using the tools specified by the silicon vendor. You then start development work on the OAL portion of the BSP. Part of the OAL work consists of implementing the bootstrapping kernel functions required to start the Windows CE kernel. Developing the OAL is a step-by-step process of implementing the startup functions and implementing the code that will initialize the hardware on the device in preparation for running the Windows CE kernel. You can leverage or share many of the startup functions that are implemented in the boot loader during OAL development. Once you have developed, downloaded, and debugged the OAL, a minimal Windows CE kernel will be running on the hardware platform.

Next, you add support for the peripherals that you have chosen for the device. To do this, you have to iteratively add and debug the drivers for these peripherals one at a time. If you have multiple developers working on drivers, the drivers can be developed in a series and added to the BSP one at a time. Depending on what



peripherals you have chosen, or if you are using a System-On-Chip (SOC), you can add drivers from Microsoft or from a third party. You can leverage these drivers as the starting point for each driver that you develop.

Once the device drivers have been developed, you should have a BSP with an OAL and the drivers for each peripheral that you intend to support on the hardware platform. Next, you will plan and implement power management.

Power management is an important part of the driver and BSP development process. Once you have planned your power management system, which includes deciding to what extent your device will rely on battery power, you will use the Power Manager to help you implement those power management capabilities. Although power management is not the focus of this article, you can find documentation that explicitly covers power management in Windows CE Help and the MSDN Library. There you will find details about how to make each peripheral driver power-management-aware, as well as how drivers interact with the Power Manager and OAL.

As you develop the OAL, bring up device drivers, and implement power management, you must test each component. The Windows CE Test Kit (CETK) provides a wide variety of tests to help you with this process. For information about the CETK, see Windows CE Help and the MSDN Library.

The final steps in the BSP development process are to create a software development kit (SDK) and package the BSP into an .msi install file so that it can be installed by others. An SDK is a set of headers, libraries, connectivity files, run-time files, OS design extensions, and Help documentation that developers use to write applications for a specific OS design. The contents of an SDK allow developers to create and debug an application on the run-time image built from your OS design. Windows CE provides an SDK Wizard to create an SDK from your BSP and an Export Wizard to package your BSP into an .msi install file.

For detailed information on all aspects of the BSP development process, see Windows CE documentation.

3. Boot Loader

3.1. Boot Loader Startup Sequence

Boot Loader Startup Sequence			
Startup()			
KernelStart()			
	Main()		
		BootloaderMain()	
		OEMDebugInit()	
		OEMPlatformInit()	
		OEMPreDownload()	
			Download occurs
		OEMLaunch()	

[Table - 1] Boot Loader Startup Sequence

1) Startup() -> platform/target/eboot/arm/fw.s

The boot loader startup function is the first code that is run on the device after a hardware reset or run-time reset, and when the system wakes up. This function performs the following tasks:

- Sets the device to supervisor mode
- Performs the necessary initialization for the following hardware:
CPU / Memory controller / System clocks / Serial / Caches / TLBs

In the case of a wakeup event, the startup function reads the saved address for the wakeup event, and then jumps to that location. If it is not a wakeup event, the boot loader is copied to memory and run.

The startup code is typically located in the Startup.s file located in the %_WINCEROOT%\Platform\<Hardware Platform Name>\Src\Bootloader or %_WINCEROOT%\Platform\<Hardware Platform Name>\Src\Bootloader\Eboot directory. The startup code is typically in assembly language because it is the first code that runs on the device and needs to do low-level hardware initialization and communication with the CPU.

Because each CPU and hardware platform requires different initialization, you must modify the Startup.s file, depending on the CPU you are using. The silicon vendor provides the CPU initialization information. The startup code can be shared with the OAL and can be reused later during that part of the development process.



2) Main() -> main.c

Depending on the CPU you are using for BSP development, Main is typically the entry point for the C code to be run. Main is typically located in the Main.c file in the %_WINCEROOT%\Platform\<Hardware Platform Name>\Src\Bootloader directory or in the %_WINCEROOT%\Platform\<Hardware Platform Name>\Src\Bootloader\Eboot directory. Although Main.c can perform any initializations on the device, it should end with a call to the BLCOMMON library entry point, BootloaderMain.

3) BootloaderMain() -> Blcommon.c

The BLCOMMON library entry point is the BootloaderMain function. BootloaderMain is the primary call in the BLCOMMON library that implements a common and consistent framework for a boot loader. The BLCOMMON library source file is located in the Blcommon.c file in the %_WINCEROOT%\Public\Common\Oak\Drivers\Ethdbg directory.

The BLCOMMON library provides the following functionality:

- Relocates the boot loader to RAM for faster execution
- Decodes the .bin file contents
- Verifies checksums
- Keeps track of load progress

The BLCOMMON library calls well-defined OEM functions throughout the process to handle hardware platform or solution-specific customizations.

4) OEMDebugInit() -> Main.c

OEMDebugInit is the first function called from the BootloaderMain function when the boot loader is started. This call is typically used to initialize serial UART debugging output. The OEM often chooses to call the OEMInitDebugSerial function to initialize a serial UART for debugging messages.

After OEMDebugInit completes, the Windows CE banner appears indicating that this interface is available. The OEM function interface can use OEMWriteDebugString, OEMWriteDebugByte, and OEMReadDebugByte after OEMDebugInit completes.

The OAL and the boot loader can share this implementation.

OEMDebugInit is located in the Main.c file in the %_WINCEROOT%\Platform\<Hardware Platform Name>\Src\Bootloader



OR %_WINCEROOT%\Platform\<Hardware Platform Name>\Src\Bootloader\Eboot directory.

5) OEMPlatformInit() -> Main.c

OEMPlatformInit is the next function called from BLCOMMON after the OEMDebugInit function completes and the Windows CE banner is displayed.

OEMPlatformInit is an OEM hardware platform initialization function for the clock, PCI interface, or NIC interface. OEMPlatformInit is also where the interruption in the boot up process takes place to display the boot loader menu. The NIC interface for downloading the image is assumed for the following download functions.

OEMPlatformInit is located in the Main.c file in the %_WINCEROOT%\Platform\<Hardware Platform Name>\Src\Bootloader or %_WINCEROOT%\Platform\<Hardware Platform Name>\Src\Bootloader\Eboot directory.

6) OEMPreDownload() -> Main.c

OEMPreDownload is also called from BLCOMMON before actually downloading a run-time image. This function retrieves the IP address for the device and connects to the development workstation. You can customize OEMPreDownload to prompt the user for feedback to obtain a static IP address or skip the download and jump to an image that might already exist on the device. The preferred method is to use the settings provided in Platform Builder.

Return values for OEMPreDownload are typically obtained through a call to EbootInitEtherTransport, which retrieves this information from Platform Builder. You can select downloading or booting from a resident run-time image by options passed from Platform Builder to the boot loader through EbootInitEtherTransport. You can also set up OEMPreDownload to read hardware switch settings to determine the code that should be returned.

The following tables shows the possible return codes for the OEMPreDownload function.

Return code	Description
BL_DOWNLOAD(0)	The Ethernet transport should be initialized by calling EbootInitEtherTransport.
BL_JUMP(1)	The download function in the BLCOMMON framework is bypassed.
BL_ERROR(-1)	Error condition.

[Table - 2] Return codes for the OEMPreDownload



If a signed run-time image has been downloaded, the signature is checked and verified before calling OEMLaunch.

OEMPreDownload is located in the Main.c file in the %_WINCEROOT%\Platform\<Hardware Platform Name>\Src\Bootloader or %_WINCEROOT%\Platform\<Hardware Platform Name>\Src\Bootloader\Eboot directory.

7) OEMLaunch() -> Main.c

OEMLaunch is the last boot loader function called out of BLCOMMON and is responsible for jumping to and launching the run-time image.

OEMLaunch can use the EbootWaitForHostConnect function, which is defined in Eboot.lib, to wait for returned arguments when you have chosen passive KITL or not to connect to the target device at boot time. Other flags are set to define the transport mechanism as a serial, USB, or Ethernet connection.

OEMLaunch jumps to the first instruction specified by the *dwLaunchAddr* parameter, which is the location of the run-time image Starup function. Starup calls the KernelStart function, as defined below. The launch address is obtained from the .bin file and is stored in the length field of the last block.

At this point, the boot loader is done executing code and the Starup function in the OAL is called.

OEMLaunch is located in the Main.c file in the %_WINCEROOT%\Platform\<Hardware Platform Name>\Src\Bootloader or %_WINCEROOT%\Platform\<Hardware Platform Name>\Src\Bootloader\Eboot directory.

4. Kernel

4.1. Kernel Startup Sequence

The startup function of the kernel is started by the OEMLaunch() function of a boot loader.

Kernel Startup Sequence			
Startup()			
	KernelStart()		
		ARMInit()	
			OEMInitDebugSerial()



			OEMInit()
		KernelInit()	
			HeapInit()
			InitMemoryPool()
			ProcInit()
			SchedInit()
	FirstSchedule()		
		SystemStartupFunc()	
		IOCTL_HAL_POSTINIT	

[Table - 3] Kernel Startup Sequence

1) Startup() -> Startup.s

The kernel Startup function is the first code that runs after the boot loader jumps to the run-time image that is now on the device, or when the development boot loader is removed and a production boot loader is flashed to the device and kernel Startup is called from the production boot loader.

Although in this example kernel Startup is a different source file than the boot loader Startup function, they have similar code and can be shared. Startup is responsible for detecting a hardware or run-time reset or detecting when the system wakes up. The kernel Startup function performs the necessary hardware initialization, including initializing the CPU, memory controller, system clocks, serial port, and caches, and detecting when the system wakes up.

Kernel Startup code is typically located in the Startup.s file in the %_WINCEROOT%\Platform\<Hardware Platform Name>\Src\Kernel\OAL directory. This directory can also share the boot loader Startup.s code. The Startup code is typically in assembly language because this is the first code that runs on the device and needs to perform low-level hardware initialization.

Because each CPU and hardware platform requires different initialization, you must modify the file containing the kernel Startup function depending on the CPU you are using. The silicon vendor provides this CPU initialization information.

Kernel Startup is responsible for computing the physical address for the OEMAddressTable and loading it into memory for use by the KernelStart function. The Startup function sets up any hardware platform or CPU-specific configurations that are required for the kernel to access ROM and DRAM, and then jumps to KernelStartup. The Startup routine should initialize any processor-specific cache or memory management unit (MMU), but not enable them.

2) KernelStart() -> armtrap.s (private/winceos/coreos/nk/kernel/arm/)

The KernelStart function is the main entry point for the kernel. The KernelStart function initializes the first-level page table based upon the contents of the MemoryMap array and enables the MMU and caches. Kernel symbolic addresses are not valid and must be translated through the MemoryMap array to find the correct physical address until the MMU is enabled. KernelStart also initializes the stacks for each mode of operation, and initializes global data for the kernel.

The KernelStart function is located in the Armtrap.s file in the %_WINCEROOT%\Private\Winceos\Coreos\Nk\Kernel\ARM directory.

3) ARMInit() -> mdarm.s (private/ directory)

The ARMInit function, which is the main ARM initialization function, is responsible for calling the OEM-supplied initialization functions. ARMInit makes the following function calls:

- Calls OEMInitDebugSerial to initialize the debug serial port through OEMInitDebugSerial
- Uses OEMWriteDebugString to display the Windows CE banner
- Calls OEMInit to perform hardware platform initialization

The ARMInit function is located in the Mdarm.c file in the %_WINCEROOT%\Private\Winceos\Coreos\Nk\Kernel\ARM directory.

4) OEMInitDebugSerial() -> mdarm.s (private/ directory)

The OEMInitDebugSerial function initializes the debug serial port on the device. The debugging functions are: OEMDebugInit, OEMWriteDebugString, OEMWriteDebugByte, and OEMReadDebugByte.

The code for OEMInitDebugSerial is very similar to the OEMDebugInit function called from the boot loader. Therefore, most of the OEMDebugInit code can be reused or shared.

5) OEMInit() -> Init.c

The kernel calls OEMInit after it has performed minimal initialization in ARMInit. When OEMInit is called, interrupts are disabled and the kernel is unable to handle exceptions. OEMInit initializes the following hardware:

- Cache / Interrupts / Clock / All other board-level hardware peripherals needed to support the hardware platform.

6) KernelInit() -> kwin32.c (private/ diretory)

The KernelInit function is where the kernel is initialized. KernelInit calls functions to initialize the following kernel components:

- Heap
- Memory pool
- Kernel process
- Scheduler

When KernelInit returns, the kernel is ready to schedule the first thread through SystemStatupFunc.

The KernelInit function can be found in the Kwin32.c file in the %_WINCEROOT%\Private\Winceos\Coreos\Nk\Kernel directory.

7) HeapInit() -> Heap.c (private/ directory)

The HeapInit function initializes the kernel heap and is located in the Heap.c file in the %_WINCEROOT%\Private\Winceos\Coreos\Nk\Kernel directory.

8) InitMemoryPool() -> physmem.c (private/ directory)

The InitMemoryPool function initializes the kernel memory pool and is located in the Physmem.c file in the %_WINCEROOT%\Private\Winceos\Coreos\Nk\Kernel directory.

9) ProcInit() -> Schedul.c (private)

The ProcInit function initializes the kernel process and is located in the Schedule.c file in the %_WINCEROOT%\Private\Winceos\Coreos\Nk\Kernel directory.

10) SchedInit() -> Scheddule.c (private/ directory)

The SchedInit function initializes the scheduler and is located in the Schedule.c file in the %_WINCEROOT%\Private\Winceos\Coreos\Nk\Kernel directory.

SchedInit creates the SystemStatupFunc thread.

11) FirstSchedule() -> Schedule.c (private/ directory)

The FirstSchedule function starts the scheduler and is located in the Schedule.c file in the %_WINCEROOT%\Private\Winceos\Coreos\Nk\Kernel directory.

12) SystemStartupFunc() -> Schedule.c (private/ directory)

The SystemStartupFunc function is called after all the required initialization has been completed and when the system is ready to schedule startup and run kernel threads. SystemStartupFunc is where calls are made to start kernel monitoring through

CreateKernelThread, and execute IOCTL_HAL_POSTINIT through the OEMIoControl function.

SystemStartupFunc is located in the Schedule.c file in the %_WINCEROOT\Private\Winceos\Coreos\Nk\Kernel directory.

5. HowTo Porting

WinCE porting divides with a boot loader and kernel.

5.1. Boot Loader Development

* WinCE5.0 basis

1) Create a directory for your new boot loader code.

Ex) %_WINCEROOT%\platform\target\ebboot\

2) Set up a command-line build shell to create and build the boot loader.

3) Generate all static libraries required by the boot loader image.

4) Create a file for your boot loader's StartUp function, and then establish a function framework as a starting point.

5) Create a sources file and makefile file in the boot loader directory.

6) Build your first boot loader executable (.exe) file.

7) Implement the boot loader's StartUp function.

8) Create a boot loader main function.

9) Create stub versions for a number of OEM functions, including generic hardware platform routines and flash memory operations.

10) Add the new source files, Main.c. and Flash.c, created in the previous step to the SOURCES line in the sources file.

11) Create the .bib file, which will be used by Romimage.exe to convert the boot loader .exe file into .bin and .nb0 files.

12) Rebuild the boot loader.

13) Implement the following serial debug functions

- OEMDebugInit
- OEMWriteDebugString
- OEMWriteDebugByte
- OEMReadDebugByte

14) Rebuild the boot loader and verify that.

15) Create the driver globals and boot args shared data areas.

16) Implement the remaining hardware platform initialization code by implementing

the [OEMPlatformInit](#) function..

17) Add Ethernet debug libraries that have routines referenced in OEMPlatformInit to the TARGETLIBS entry in the boot loader sources file.

18) Implement the following Ethernet controller-related functions.

- OEMReadData
- OEMEthGetFrame
- OEMEthSendFrame
- OEMEthGetSecs

19) Implement the [OEMPreDownload](#) function.

20) Test boot loader communications.

21) Create a test .bin file to download from Platform Builder.

22) Verify that the boot loader can use Platform Builder to download a .bin file.

23) Implement the [OEMLaunch](#) function.

24) Implement the [OEMIsFlashAddr](#) function.

25) Implement the [OEMMapMemAddr](#) function.

26) Implement the following flash memory erase functions.

- OEMStartEraseFlash
- OEMContinueEraseFlash
- OEMFinishEraseFlash

27) Implement the [OEMWriteFlash](#) function.

28) Modify the .bib file to produce a boot loader image that downloads to flash memory.

29) To add support for download progress indication, implement the [OEMShowProgress](#) function.

30) To verify the RAM or flash memory used by the run-time image before the run-time image is downloaded completely, add support for memory verification.

31) If you are developing a multi-BIN run-time image and you would find multi-BIN run-time image notification useful, add support for multi-BIN run-time image notification.

32) To have your boot loader validate the run-time image before downloading or running it, add support for run-time image signatures.

5.2. Kernel Development

* WinCE basis

1) Create a hardware platform directory and a subdirectory for the OAL on your development workstation.

2) Implement the Startup function for the OAL.

-
- 3) Create a sources file and a makefile to assemble and compile the file containing the StartUp function.
 - 4) Build the StartUp source file.
 - 5) Create the Kernel directories and create a dirs file to direct the build process.
 - 6) Create stub versions of the following CPU-specific OAL functions.
 - OEMARMCacheMode
 - OEMDataAbortHandler
 - OEMInterruptHandler
 - OEMInterruptHandlerFIQ
 - 7) Create stub versions of the following required OAL functions.
 - InitClock
 - OEMInit
 - OEMCacheRangeFlush
 - OEMGetExtensionDRAM
 - 8) Create stub versions of the following functions.
 - OEMDebugInit
 - OEMInitDebugSerial
 - OEMReadDebugByte
 - OEMWriteDebugByte
 - OEMWriteDebugString
 - 9) Create stub versions of the following functions.
 - OEMIdle
 - OEMPowerOff
 - 10) Create stub versions of the following interrupt functions.
 - OEMInterruptDisable
 - OEMInterruptDone
 - OEMInterruptEnable
 - SC_GetTickCount
 - 11) Create stub versions of the following real-time clock (RTC) functions.
 - OEMGetRealTime
 - OEMSetAlarmTime
 - OEMSetRealTime
 - 12) Create a stub versions of the following function.
 - OEMIoControl
 - 13) Define the following global variables required by Nk.lib.
 - OEMAddressTable
 - 14) Build the kernel executable image, Kernkitl.exe, when stub versions of all the functions and global variables are defined.
-



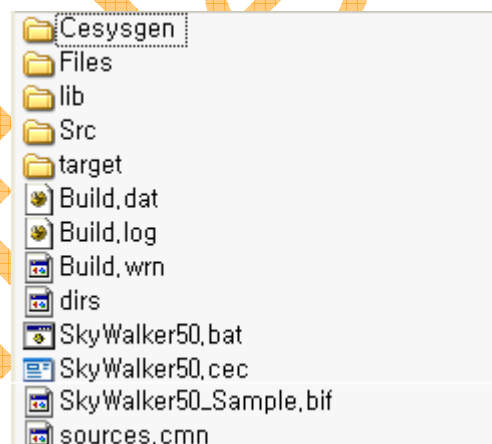
- 15) After creating Kernkitl.exe, you can start adding functionality to the run-time image in stages.
- 16) Implement the following CPU-specific pre-OEMInitDebugSerial functions and global variables.
 - OEMAddressTable
 - OEMARMCacheMode
 - OEMDataAbortHandler
- 17) Implement the [OEMCacheRangeFlush](#) function.
- 18) Stop and ensure that the run-time image builds.
- 19) Create a %_WINCEROOT%\Platform\MyPlatform\Files directory and create the following empty files in the new directory: Platform.bib, Platform.reg, Platform.db, and Platform.dat.
- 20) Create a %_WINCEROOT%\Platform\MyPlatform\Cesysgen directory and copy the makefile file from a similar hardware platform into that directory.
- 21) Ensure that all appropriate hardware platform directories have dirs files.
- 22) From the command line, build an Nk.bin image by entering the following command.
- 23) Verify that you can download the Nk.bin image and that you can boot up to at least the OEMInitDebugSerial function in the OAL using any of the following debugging tools: hardware debugger or probe, LED write, or serial debug prints.
- 24) Implement the following serial debug functions. These are the same routines used by the boot loader so you can share the code with the OAL.
 - OEMInitDebugSerial
 - OEMReadDebugByte
 - OEMWriteDebugByte
 - OEMWriteDebugString
- 25) Rebuild Nk.bin and verify that the boot process proceeds through OEMInit.
- 26) Implement the [OEMInit](#) function.
- 27) Implement the interrupt-related code.
 - OEMInterruptEnable
 - OEMInterruptDisable
 - OEMInterruptDone
 - OEMInterruptHandler
 - OEMInterruptHandlerFIQ
- 28) Implement the power management functions.
 - OEMIdle
 - OEMPowerOff
- 29) Rebuild the Nk.bin image and verify that it boots successfully.

- 30) Add KITL initialization code to OEMInit.
- 31) Add the Windows CE Target Control Shell to your run-time image. At the build shell prompt, enter the following command.
- 32) Rebuild the Nk.bin image and verify that it boots successfully. Ensure that Platform Builder can connect over KITL to the target using the Windows CE Target Control.
- 33) Implement the following RTC functions.
 - OEMGetRealTime
 - OEMSetAlarmTime
 - OEMSetRealTime
- 34) Customize memory usage by doing the following tasks.
- 35) Implement the [OEMIoControl](#) function.

5.3. PLATFORM\SkyWalker50\ directory

5.3.1. Directory Overview

PLATFORM\SkyWalker50\ 디렉토리의 전체적인 구조는 다음과 같다.



[Fig - 3] PLATFORM\SkyWalker50\ directory

Cesysgen 디렉토리와 Files 디렉토리는 생성하고자 하는 BSP의 메모리 및 모듈 설정 파일들이 있다.

Lib 디렉토리는 BSP 빌딩 시 생성되는 라이브러리들이 위치한다.

Target 디렉토리는 BSP 빌딩 시 생성되는 모듈들(.dll or .exe)이 위치한다.

실제로 ebootloader와 kernel, 드라이버 소스들은 PLATFORM\SkyWalker50\ 디렉토리에서 존재한다.

5.3.2. Sub-directory and Files

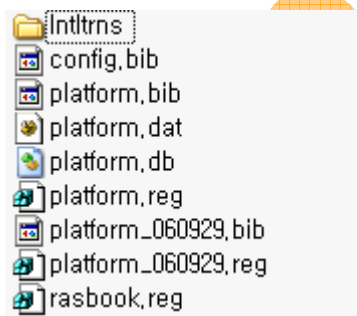
1) Src\Cesysgen\files\ directory



[Fig - 4] Src\Cesysgen\files directory

- config.bib : Memory area define
- platform.bib : Platform specific module allocation define
- platform.dat
- platform.db
- platform.reg : Top level registry define

2) Src\Files\ directory



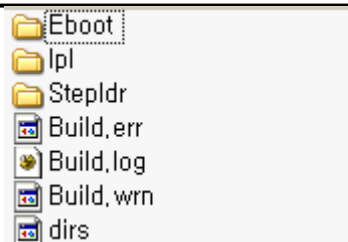
[Fig - 5] Src\Files directory

- Intlnrs\platform.str: Language support string file
- config.bib : Memory area define
- platform.bib : Platform specific module allocation define
- platform.dat
- platform.db
- platform.reg : Registry information (Cesysgen\의 reg file과 SDHC/PM 틀림)
- rasbook.reg

3) Src\lib\ directory

Target platform의 프로젝트 컴파일 시 생성되는 라이브러리를 저장한다.

4) Src\Bootloader\ directory



[Fig - 6] Src\Bootloader\ directory

Src\Bootloader\Eboot\

- boot.bib : Bootloader memory area define
- sources : source hierarchy files
- bitmap.c : Just bitmap array file
- debug.c : OEMWriteDebugString function define
- ether.c : Ethernet OEM function file
- flash.c : Flash OEM function file
- loader.h : Bootloader memory configuration define
- main.c : Bootloader main file
- makefile.inc
- nand.cpp : Nand flash manipulate function file
- starup.s : Bootloader startup code
- util.s

Src\Bootloader\Stepldr\

- stepldr.bib : Steppingstone loader bib file
- main.c : Main file
- makefile.inc
- memcfg.in : Memory bank configuration file
- nand.c : Nand flash file
- option.h : OFFSET address define
- option.inc : Memory bank configuration file for .s file
- skywalkeraddr.h : Register map define
- skywalkeraddr.inc : Register map for .s file define
- starup.s : Startup code
- utils.c : Uart util file

5) Src\Common\ directory



[Fig - 7] Src\Common\ directory

Src\Common\Smartmedia\Dll\

- smflash.def : Just export flash driver function

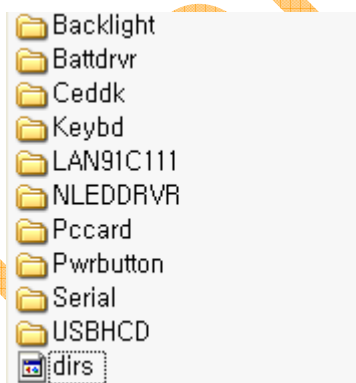
Src\Common\Smartmedia\Fmd\

- fmd.cpp : Nand flash manipulate define
- nand.h : Nand flash macro define

Src\Common\Smartmedia\Fmd\Arm\

- nand.s : Nand flash related routines

6) Src\Drivers\ directory



[Fig - 8] Src\Drivers\ directory

Src\Drivers\Backlight\

- Bak_hw.cpp : Backlight manipulate function define
- Bak_hw.h
- Bak_io.cpp : Backlight driver function define

Src\Drivers\Battdrvr\

- battdrvr.reg : Battery driver registry
- battery.h : Battery structure define
- battif.c : Battery PDD function define

Src\Drivers\Ceddk\Ddk_bus\

- data.c : Data get/set hal function define



- trans.c : Address translate function define

Src\Drivers\Ceddk\Dll\

- ceddk.c : Just ceddk entry function define

Src\Drivers\LAN91C111\

- LAN91C111.def
- LAN91C111.reg
- LAN91c111_Adapter.h
- LAN91C111_Filter.c
- LAN91C111_Init.c
- LAN91C111_Intr.c
- LAN91C111_Miniport.c
- LAN91C111_NDIS.c
- LAN91C111_Phy.c
- LAN91C111_Proto.h

Src\Drivers\Keybd\Kbdcommon\

- kbd.cpp : Driver function define
- smt926kbd.cpp : Keyboard manipulate function define
- smt926kbd.hpp : Keyboard class define

Src\Drivers\Keybd\Kbdsmt926jpn1\

- kbdjpn1.def : Just export the driver function

Src\Drivers\Keybd\Kbdsmt926jpn2\

- kbdjpn2.def : Just export the driver function

Src\Drivers\Keybd\Kbdsmt926kor\

- kbdkor.def : Just export the driver function

Src\Drivers\Keybd\Kbdsmt926us\

- kbdus.def : Just export the driver function

Src\Drivers\Keybd\Matrix_0409\

- smt926.cpp : Keyboard PDD function define

Src\Drivers\Keybd\Pddlist\

- pddlist.cpp : Key board entry point declare

Src\Drivers\NLEDDRV\

- nled.cpp : LED PDD function define
- nleddrvr.reg

Src\Drivers\Pccard\

- pcc_skywalker.reg
- init.cpp : Defin platform dependent PCMCIA initialization functions
- pcmsock.cpp : Define pcmcia socket functions
- pcmsock.h : PCMCIA register define
- pcmwin.cpp : PCMCIA Memory and IO Window functions
- pd6710.h
- pdsocket.cpp : PDD initialization functions
- pdsocket.h : PDD class define

Src\Drivers\Pwrbutton\

- pwrbtnskywalker.c : Power button driver function define
- pwrbtnskywalker.h : Power button structure define

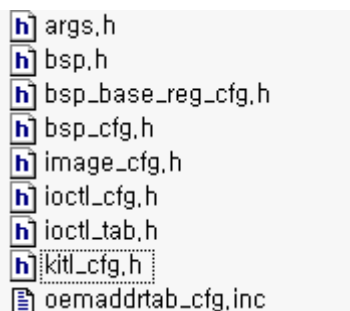
Src\Drivers\Serial\

- ser_skywalker.cpp : Serial PDD function define

Src\Drivers\USBHCD\

- skywalker_ohci.c : USB OHCD function define

7) Src\Inc\ directory

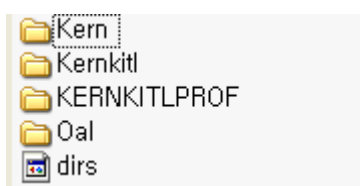


[Fig - 9] Src\Inc\ directory

- args.h : BSP arg structure define
- bsp.h : Include the bsp header files

- bsp_base_reg_cfg.h : Define the bsp base register address
- bsp_cfg.h : BSP configuration macro define
- image_cfg.h : Define the image configuration address
- ioctl_cfg.h : Define a configuration for IOCTL component
- ioctl_tab.h : Configuration file for the OAL IOCTL component
- kitl_cfg.h : Serial or Ethernet kitl macro & structure define
- oemaddrtab_cfg.inc : Define global oal address table

8) Src\Kernel\ directory



[Fig - 10] Src\Kernel\ directory

Src\Kernel\Kern\

- kitl.c : Kitl function define

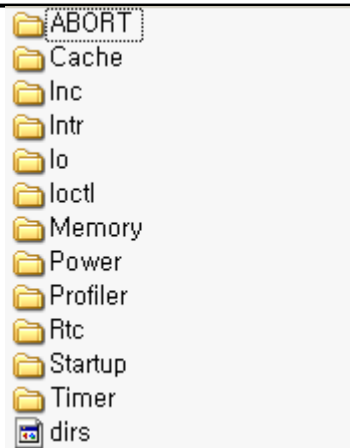
Src\Kernel\Oal\

- args.c : Define OALArgsQuery function
- bitmap.c : Just bitmap array file
- debug.c : OEM serial function file
- init.c : Board initialization code file
- intr.c : BSP interrupt code file
- ioctl.c : OEM IOCTL variable declare
- kitl.c : KITL initialization function define
- kitlser.c : Serial KITL function define
- kitlusbser.c : USB serial KITL function define
- power.c : BSP power on/off function define
- startup.s : Kernel startup code

5.4. PLATFORM\COMMON\ directory

5.4.1. Directory Overview

PLATFORM\COMMON\SRC\ARM\SMT\SMT926A\ 디렉토리의 전체적인 구조는 다음과 같다.



[Fig - 11] SRC\ARM\SMT\SMT926A\ directory

5.4.2. Sub-directory and Files

ABORT\	
- sources	: Just using oal_abort_arm920t.lib
Cache\	
- sources	: Just using oal_cache_arm926t.lib
Inc\	
- intr.h	: Just Profiler ISR function pointer define
Intr\	
- fiq.c	: FIQ handler function define
- intr.c	: IRQ handler function define
- sources	: Using oal_intr.lib
Io\	
- sources	: Just using oal_io.lib
Ioctl\	
- reboot.c	: Define the OALIoCtlHalReboot function
- sources	: Using oal_ioctl.lib
Memory\	
- sources	: Just using oal_memory_arm.lib
Power\	
- off.c	: Define OEMPoweroff function
- sources	: Using oal_power.lib
Profiler\	
- profiler.c	: Define profiler relative functions
Rtc\	
- rtc.c.	: Define OEM function relative to RTC
Startup\	

- starup.s : Startup code for SMT926

Timer\

- sources : Using oal_timer.lib and oal_timer_varidle.lib

- timer.c : Define the OALCPUIdle function

- watchdog.c : Define the function for WDT

5.5. PUBLIC\COMMON\ directory

5.5.1. Directory Overview

PUBLIC\COMMON\OAK\CSP\ARM\SMT\SMT926A\ 디렉토리의 전체적인 구조는 다음과 같다.



[Fig - 12] OAK\CSP\ARM\SMT\SMT926A\ directory

5.5.2. Sub-directory and Files

Inc\

- pddsmt926_ser.h : Platform dependent serial definition for serial controller

- smt926a.h : Just smt926 peripheral register define header file include

- smt926a.inc : Header file to startup code

- smt926a_base_regs.inc : Include the smt926a_base_regs.inc file

- smt926a_adc.h : Header file to ADC

- smt926a_base_regs.h

: Header file to SMT926 chip's base controller register physical addr

- smt926a_base_regs.inc: File included by start-up code

- smt926a_cam.h : Header file to the camera I/F

- smt926a_dma.h : Header file to define the DMA controller register

- smt926a_dmatransfer.h: Header file to define the DMA register bit allocation

- smt926a_iicbus.h : Header file to define the I2C bus control register

- smt926a_iisbus.h : Header file to define the I2S bus control register

- smt926a_intr.h : Header file to define the Interrupt control register



- smt926a_ioport.h : Header file to define the I/O port control register
- smt926a_lcd.h : Header file to define the LCD control register
- smt926a_memctl.h : Header file to define the memory controller register
- smt926a_nand.h : Header file to define the NAND flash controller register
- smt926a_pwm.h : Header file to define the PWM timer control register
- smt926a_rtc.h : Header file to define the RTC control register
- smt926a_sdi.h : Header file to define the SDI control register
- smt926a_spi.h : Header file to define the SPI control register
- smt926a_uart.h : Header file to define the UART controller register
- smt926a_usbd.h : Header file to define the USB control register
- smt926a_wdog.h : Header file to define the Watchdog timer register

DISPLAY\SMT926A_LCD\

- precomp.h : Just include the header files
- smt926a_lcd.cpp : Source file to LCD display
- smt926a_lcd_def : File to export LCD display function
- smt926a_lcd.h : Header file to LCD display function

SDHC\SDHC\

- sdhc_smt026.def : Just file to export SDIO function
- sdioccontroller.cpp : Define the SDIO function
- sdioccontroller.h : Header file to define SDIO controller class

SDHC\SDHCBASE\

- sdhcmain.cpp : Driver function file to SD
- sdioccontrollerbase.cpp: Source file to be defined detail functions for SD
- sdioccontrollerbase.h : Header file to define the Register bit allocation and class for SD

Serial\

- pddsmt926_ser.cpp : Serial PDD for UART Common code

Touch\

- smt926a_touch.cpp : Platform dependent touch initialization functions

Usb\Function\

- smt926pdd.cpp : Source code for USB function
- smt926pdd.h : USB function PDD header

- smt926usbfndef : Just file to export USB function

Wavedev\

- audiosys.h : Header file
- devctxt.cpp : Device context file
- devctxt.h : Device context class define header
- hwctxt.cpp : Platform dependent code for the mixing audio driver
- hwctxt.h : Header file to define PDD class
- i2s.cpp : I2S I/F routines definition
- i2s.h : I2S bit allocation definition
- input.cpp : Input stream manipulate function
- midinode.cpp
- midistrm.cpp : Midi (Audio-codec) control function define
- modistrm.h : Midi class define
- mixerdrv.cpp : Audio mixer driver function define
- mixerdrv.h
- output.cpp : Output stream manipulate function
- strmctxt.cpp : Stream manipulate function
- strmctxt.h : Stream class define
- wavemain.cpp : File to be defined functions for wave driver
- wavemain.h : Header file to be included using header file and class
- wavepdd.h : Wave PDD structure define
- wfmtmidi.h : Wave format structure define

5.6. Functional and Source code porting

5.6.1. PLATFORM\SkyWalker50\ directory

PLATFORM\SkyWalker50\

File Name	Porting	Remark
SkyWalker50.bat	☐	
sources.cmn	☐	

Src\Bootloader\Eboot\

File Name	Porting	Remark
boot.bib	☐	Further work
sources	☐	
bitmap.c	☐	Screen 변경 시 수정 필요
debug.c	☐	
ether.c	☐	LAN91C 초기화 루틴 수정 필요
flash.c	☐	Further work
loader.h	☐	Further work
main.c	☐	Further work
nand.cpp	☐	상황에 따라 변경 선택

Startup.s	☒	Further work
Util.s	☒	매크로 설정만 변경 필요

* 4.2 base ebootloader source 이용 예정 (2006/10/09)

Src\Bootloader\Stepldr\

File Name	Porting	Remark
main.c	☒	(Further work)Macro 수정 필요
memcfg.inc	☒	Further work
nand.c	☒	Further work
option.h	☒	(Further work)Macro 수정 필요
option.inc	☒	(Further work)Macro 수정 필요
SkyWalkeraddr.h	☒	Further work
Skywalkeraddr.inc	☒	Further work
startup.s	☒	Further work
Stepldr.bib	☒	Further work
utils.c	☒	Further work

Src\Common\Smartmedia\Fmd\

File Name	Porting	Remark
Fmd.cpp	☒	Further work
nand.h	☒	Further work
nand.s	☒	Further work

Src\Drivers\

Src\Inc\

File Name	Porting	Remark
args.h	☐	포팅 할 필요 없음
bsp.h	☐	포팅 할 필요 없음
bsp_base_regs_cfg.h	☒	Further work
bsp_cfg.h	☒	Further work
image_cfg.h	☒	Further work
ioctl_cfg.h	☒	일단 그대로 사용
ioctl_tab.h	☒	일단 그대로 사용 Further work
kitl_cfg.h	☒	Further work
oemaddrtab_cfg.inc	☒	Further work

Src\Kernel\Kern\

File Name	Porting	Remark
kitl.c	☒	Further work???

Src\Kernel\Oal\

File Name	Porting	Remark
args.c	☐	그대로 사용
bitmap.c	☐	Screen 변경 시 수정 필요
debug.c	☒	Further work
init.c	☒	Further work
* intr.c	☒	Further work
ioctl.c	☒	일단 그대로 사용
* kitl.c	☒	Further work
kitlser.c	☒	Further work
kitlusbser.c	☒	Further work
power.c	☒	Further work
startup.s	☒	Further work

- Don't need to porting
- Need to porting
- If needs, porting

6. Source Analysis

실제로 porting 과정에서 eBoot loader와 Kernel의 각 파일들 소스를 분석하도록 한다.

6.1. Cache Function

S3c2440 에서 ARM920T를 사용하는 반면 SkyWalker는 ARM926EJ-S를 사용한다. 이 경우 cache 사용에 있어 co-processor cp15 와 보조 레지스터 명령어 사용은 동일하다. 그러나, I/D cache의 flush/clean과 lock down 시 ARM core의 GPR 레지스터에 의한 설정 값 전달 시 레지스터 값의 포맷은 조금씩 다르다. (SciTech사의 ARM System Developer's Guide 참조)

OEMCacheRangeFlush()
OALIoCtlHalGetCacheInfo()
OALCacheGlobalsInit()

Function Name	Source Location
OALFlushDCache	ARM926\CACHE\flushdc.s
OALFlushICache	COMMON\CACHE\flushic.s
OALFlushDCacheLines	COMMON\CACHE\flushdclines.s
OALFlushICacheLines	COMMON\CACHE\flushiclines.s
OALCleanDCache	ARM926\CACHE\cleandc.s
OALCleanDCacheLines	COMMON\CACHE\cleandclines.s

- * PLATFORM\COMMON\SRC\ARM directory 기준
- * Refer to PLATFORM\COMMON\SRC\INC\oal_cache.h
- * OALFlushDCacheLines/OALFlushICacheLines/OALCleanDCacheLines
(void *pAddress, UINT32 size)

[Table - 4] Cache OAL 함수

6.2. MMU Function

ARM920T와 ARM926EJ-S core의 경우 동일한 MMU 기능을 가지며 MMU 관련 cp15:cx 레지스터 포맷 또한 동일하다.

Function Name	Source Location
OALClearDTLB	COMMON\CACHE\cleardtlb.s
OALClearITLB	COMMON\CACHE\clearitlb.s
OALClearUTLB	COMMON\CACHE\clearutlb.s
OALClearDTLBEntry	COMMON\CACHE\cleardtlbentry.s
OALClearITLBEntry	COMMON\CACHE\clearitlbentry.s
OALClearTLBEntry	

* PLATFORM\COMMON\SRC\ARM directory 기준

* Refer to PLATFORM\COMMON\SRC\INC\oal_cache.h

[Table - 5] TLB OAL 함수

6.3. Data Abort Function

Abort 관련 파일들에 대한 설명이다.

oemabort.s

dabort.s 파일을 include한다.

HandlerCallStd -> APCS_MACRO

HandlerName -> HandlerMacro

dabort.s

PLATFORM/COMMON/src/ARM/ARM920T/dabort.s와 동일하다.

ARM instruction 관련 bit field definition, register naming, register bit field definition의 어셈블리 시 독립적인 정의, 어셈블리 시 처리 과정을 위한 설정, 어셈블리 시 영향을 받는 의존적인 정의, data abort 발생 시 arm 명령어의 각 비트 필드 별 분석과 OS와 연관된 핸들러 호출, 리턴 형태, 초기화에 대해 정의 되어 있다. 실제 자세한 동작은 파악하지 못하였다.

6.4. eBoot Loader

6.4.1. Src\Bootloader\Eboot\ directory

1) ether.c

Function Name	Check point
InitEthDevice()	LAN91C111 intialize routine Smsc100FdInit()함수 인자 이해 필요
OEMGetRealTime()	RTC register mapping 필요
OEMEthGetSecs()	Don't care this function
OEMEthGetFrame()	PUBLIC\COMMON\.. smsc100.c 참조
OEMEthSendFrame()	PUBLIC\COMMON\.. smsc100.c 참조

2) flash.c

Function Name	Check point
---------------	-------------

OEMIsFlashAddr()	Check whether the addr is flash addr (추후변경)
OEMMapMemAddr()	Return FILE_CACHE_ADDR (그대로 사용)
OEMStartEraseFlash()	코드 수정 고려??? (Called by blcommon.c)
OEMContinueEraseFlash()	코드 수정 고려??? (Called by blcommon.c)
OEMFinishEraseFlash()	코드 수정 고려??? (Called by blcommon.c)
OEMWriteFlash()	코드 추가 필요 (Called by blcommon.c)

* FLIE_CACHE_START : WinCE memory architecture에서 CACHED 영역내에서 RAM을 위한 영역을 나타냄 (Refer to Src/inc/Oemaddrtab_cfg.inc)

3) loader.h

Macro Name	Check point
STEPLDR_RAM_IMAGE_BASE	Step-stone loader base addr
EBOOT_RAM_IMAGE_BASE	
EBOOT_STORE_ADDRESS	
BINFs_RAM_START	What is it???
ROM_RAMIMAGE_START	RAM area address
FILE_CACHE_START	WinCE Cached 영역의 file cache area
NBOOT_BLOCK	NAND Boot block memory map
TOC_BLOCK	
EBOOT_BLOCK	

4) main.c

Function Name	Check point
InitDisplay()	LCD controller register setting 필요
SetIP()	pBootCfg->EdbgAddr.dwIP setting(그대로 사용)
SetMask()	pBootCfg->SubnetMask setting (그대로 사용)
SetDelay()	pBootCfg->BootDelay setting (그대로 사용)
mystrtoul()	Convert string to unsigned long (그대로 사용)
CvtMAC()	Convert MAC addr (string->ul) (그대로 사용)
SetLAN91C111MACAddress()	Set Ethernet MAC addr (그대로 사용)
MainMenu()	(그대로 사용)
OEMPlatformInit()	Platform initialize (그대로 사용) Called by blcommon.c:BootloaderMain()
OEMPreDownload()	Initialize ethernet (그대로 사용) Called by blcommon.c:BootloaderMain()
OEMReadData()	Call ethernet read function (그대로 사용)
OEMShowProgress()	수정??? (그대로 사용) Called by blcommon.c:DownloadImage()
OEMLaunch()	Write image(option), Launch (그대로 사용) Called by blcommon.c:BootloaderMain()
OEMVerifyMemory()	Determine the image type (그대로 사용) Called by main.c:OEMDebugInit()
OEMMultiBINNotify()	Notify multibinary (그대로 사용) Called by main.c:OEMDebugInit()
GetKernelExtPointer()	Get kernel extention pointer (그대로 사용)
OEMDebugInit()	Initialize UART (그대로 사용) Called by blcommon.c:BootloaderMain()

5) nand.cpp

Function Name	Check point
BootConfigPrint()	(그대로 사용)
BootConfigInit()	(그대로 사용)

ID_Print()	(그대로 사용)
TOC_Print()	(그대로 사용)
TOC_Init()	(그대로 사용)
TOC_Read()	(그대로 사용)
TOC_Write()	(그대로 사용)
WriteOSImageToBootMedia()	정확한 이해 필요-Boot partition (그대로 사용) Called by main.c:OEMLaunch()
ReadOSImageFromBootMedia()	정확한 이해 필요-Boot partition (그대로 사용) Called by main.c:OEMPlatformInit()
ReadBlock()	(그대로 사용)
WriteBlock()	(그대로 사용)
WriteRawImageToBootMedia()	(그대로 사용) Called by main.c:OEMLaunch()

6) startup.s

Function Name	Check point
	많은 변경 필요 (소스 분석 필요) 4.2 version asm source 사용할 예정

7) util.s

Function Name	Check point
PHY_RAM_START macro	수정 필요
VIR_RAM_START macro	수정 필요

6.4.2. Src\Bootloader\stepldr\ directory

1) main.c

Function Name	Check point
main()	Initialize (그대로 사용)

2) nand.c

Function Name	Check point
NF_ReadPage()	Nand device spec에 맞춰 코드 수정 필요
NF_Reset()	Nand device spec에 맞춰 코드 수정 필요
NF_Init()	Nand controller initialize

3) startup.s

Function Name	Check point

4) utils.c

Function Name	Check point
Port_Init()	GPIO & EINT initialize (수정 필요)
Led_Display()	LED initialize (수정 필요)
ChangeClockDivider()	Setting clock divider (수정 필요)
ChangeMPILValue()	MPIL setting (수정 필요)
Uart_Init()	UART initialize (수정 필요)
Uart_SendByte()	Char data send (수정 필요)
Uart_SendString()	Send the string (그대로 사용)
Uart_SendDWORD()	Send dword (그대로 사용)

4.3 Src\Common\Smartmedia\Fmd\

1) fmd.cpp

Function Name	Check point
ReadFlashID()	Nand flash ID read (수정 필요)
FMD_Init()	Nand controller initialize (수정 필요)
FMD_ReadSector()	Read nand sector (수정 필요)
FMD_EraseBlock()	Erase nand block (수정 필요)
FMD_WriteSector()	Write sector (수정 필요)
FMD_PowerUp()	Nand controller initialize (수정 필요)
FMD_PowerDown()	Don't care???
FMD_OEMIoControl()	Don't care??? (Why driver func form?)
FMD_Deinit()	Don't care??? (Why driver func form?)
FMD_GetInfo()	(그대로 사용)
IsBlockBad()	(수정 필요)
FMD_GetBlockStatus()	(그대로 사용)
MarkBlockBad()	Mark the bad block (그대로 사용)
FMD_SetBlockStatus()	(그대로 사용)

2) nand.s

Function Name	Check point
LDR4STR1 macro	소스 분석 후 수정 필요
STR4LDR1 macro	소스 분석 후 수정 필요
RdPage512()	소스 분석 후 수정 필요
RdPage512Unalign()	소스 분석 후 수정 필요
WrPage512()	소스 분석 후 수정 필요
WrPage512Unalign()	소스 분석 후 수정 필요
RdPageInfo()	소스 분석 후 수정 필요
WrPageInfo()	소스 분석 후 수정 필요

6.5. Kernel

6.5.1. Src\kernel\oal\ directory

1) debug.c

Function Name	Check point
OEMInitDebugSerial()	UART Tx/Rx configure (수정 필요)
OEMWriteDebugByte()	UART char write (수정 필요)
OEMReadDebugByte()	UART char read (수정 필요)
OEMWriteDebugLED()	LED Enable (H/W 지원 시 수정)

2) init.c

Function Name	Check point
OEMInit()	수정 필요?? (그대로 사용??)
InitDisplay()	LCD controller initialize (수정 필요)
ConfigureGPIO()	GPIO initialize (수정 필요)

3) intr.c

Function Name	Check point
BSPIntrInit()	Interrupt initialize (수정 필요)
BSPIntrRequestIrqs()	소스 분석 후 수정 필요
BSPIntrEnableIrq()	수정 필요
BSPIntrDisableIrq()	수정 필요
BSPIntrDoneIrq()	수정 필요

BSPIntrActiveIrq()	수정 필요
* BSPxxx 함수 호출 관계 파악 필요	

4) kitl.c

Function Name	Check point
InitKitlEtherArgs()	Ethernet for kitl initialize (수정 필요)
InitKitlSerialArgs()	Serial for kitl initialize (수정 필요)
InitKitlUSBSerialArgs()	USB serial for kitl initialize (수정 필요)
OALKitlStart()	Kitl start (수정 필요)
OALGetTickCount()	수정 필요
SetKMode()	수정 필요

* **OAL** 관련 함수 호출 부분 및 구현 주의해야 함

5) kitlser.c

Function Name	Check point
SMT926UART_Init()	GPIO & UART reg initialize (수정 필요)
SMT926UART_WriteData()	UART char write (수정 필요)
SMT926UART_ReadData()	UART char read (수정 필요)
SMT926UART_EnableInt()	Polling???
SMT926UART_DisableInt()	Polling???
SMT926UART_PowerOff()	Disable??? (일단 그대로 사용)
SMT926UART_PowerOn()	UART initialize & Configure (수정 필요)

6) kitlusbser.c

Function Name	Check point
WaitMS()	Wait (그대로 사용)
SMT926USBSER_Init()	USB initialize (수정 필요)

7) power.c

Function Name	Check point
BSPPowerOff()	수정 필요
ConfigStopGPIO()	Where does it used??? (수정 필요)
BSPPowerOn()	Power on scenario에 맞춰 수정 필요
InitTimer()	Timer initialize (수정 필요)

* **BSP** 관련 함수 호출 부분과 구현 주의해야 함

6.6. Header Files

6.6.1. Src\Inc\ directory

1) bsp_base_reg_cfg.h

Macro Name	Check point
BSP_BASE_REG_PA_LAN91C111~	Macro define 수정 필요
BSP_BASE_REG_PA_AM29LV800	Don't need this macro

2) bsp_cfg.h

Macro Name	Check point
SMT926A_FCLK	Macro define 수정 필요
SMT926A_HCLK	Macro define 수정 필요
SMT926A_PCLK	Macro define 수정 필요
BSP_UART1 macro	Macro define 수정 필요

3) image_cfg.h

Macro Name	Check point
IMAGE_SHARE macro	Refer to config.bib (수정 필요)
IMAGE_WINCE macro	Refer to config.bib (수정 필요)

4) ioctl_cfg.h

Macro Name	Check point
IOCTL_PROCESSOR_INSTRUCTION_SET	(수정 필요)
IOCTL_PROCESSOR_CLOCK_SPEED	(수정 필요)

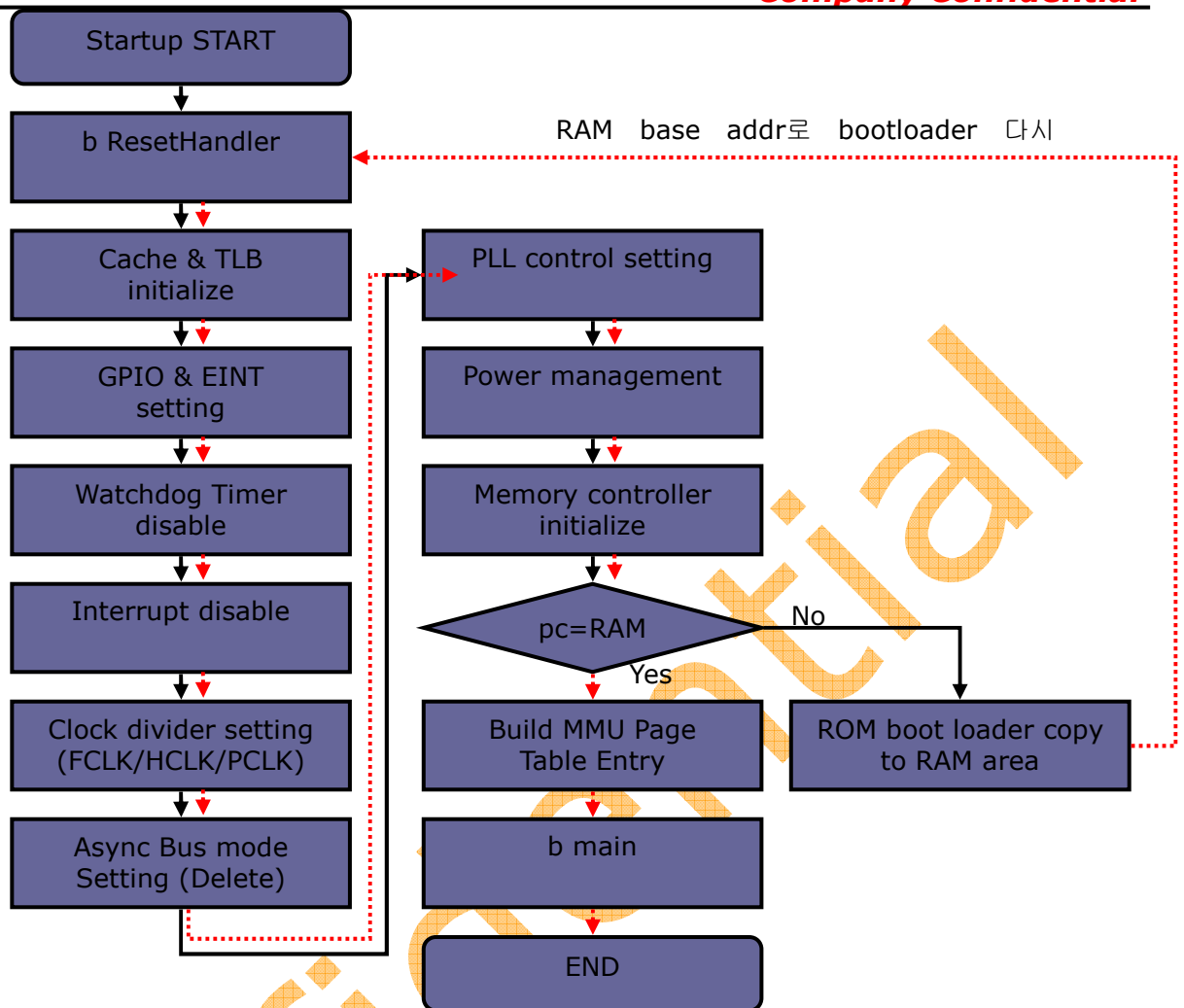
5) kitl_cfg.h

Function Name	Check point
BSP_UART macro	(수정 필요)
Kitl USB Serial macro	소스 분석 후 수정 필요(Help:이병희 선임)

6.7. Startup Code

6.7.1. Src\Bootloader\Eboot\startup.s

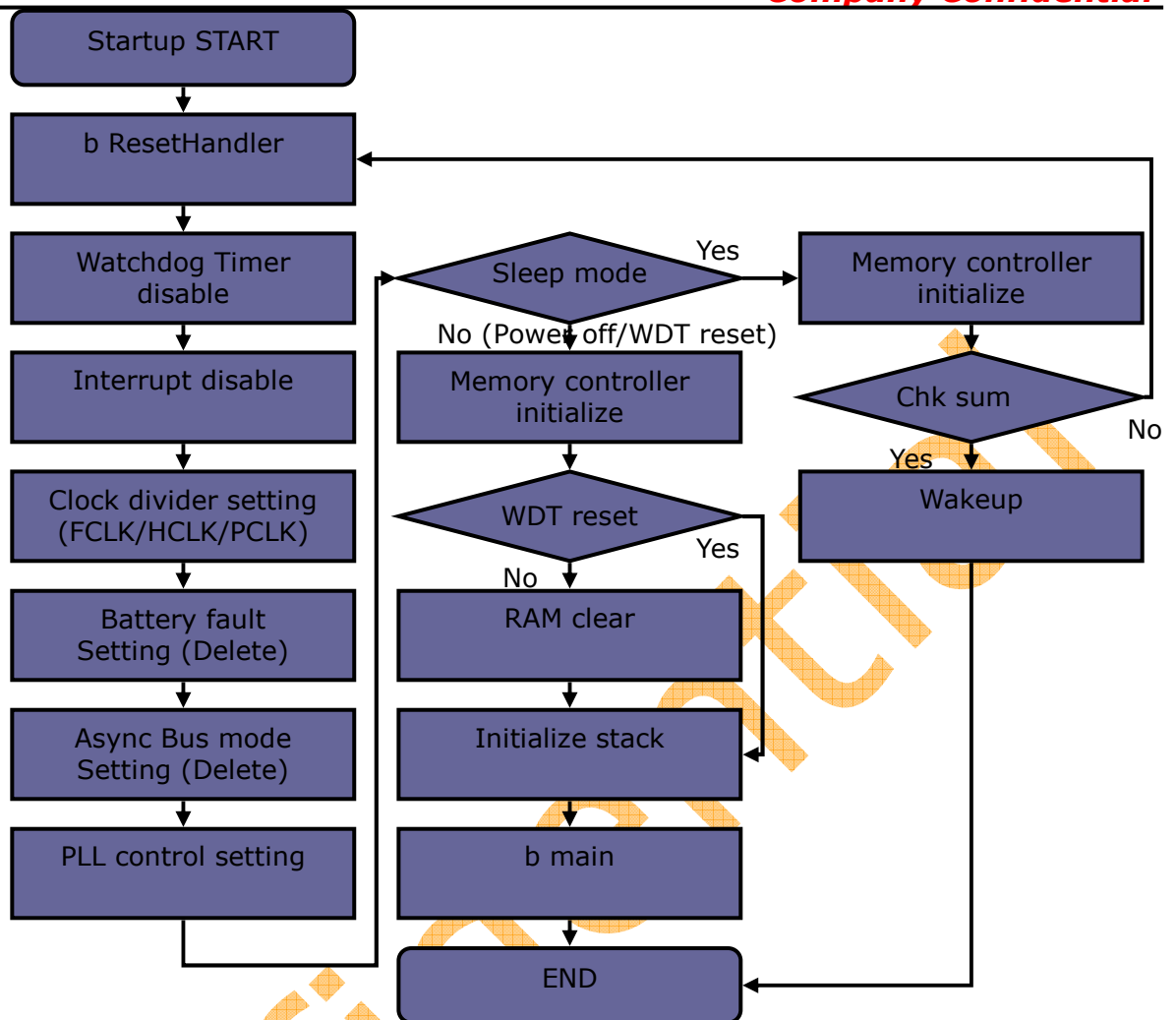
다음은 부트로더의 startup 코드의 실행 순서를 나타낸다. ROM에 존재하는 코드 수행 후 부트로더를 RAM으로 복사 후 다시 실행하며, 부트로더의 메인 함수로 점프한다. 부트로더는 NAND flash 부트 또는 ROM 부트 시 사용된다.



[Fig - 13] Bootloader startup code flow chart

6.7.2. Src\Bootloader\Stepldr\startup.s

다음 그림은 NAND flash 부팅 시 startup 코드의 실행 순서를 도시하고 있다. Power-off, watch-dog timer reset 그리고 sleep mode에서의 wake-up에 따라 실행이 틀리다. Stepstone loader(NAND flash 부팅 시 사용됨)의 메인 함수 호출 후 메인 함수에서 부트로더를 RAM 영역으로 복사하여 부트로더를 실행한다. (Sleep mode에서 wake-up 경우는 제외)



[Fig - 14] Step-stone loader startup code flow chart

C-level 함수 호출을 위해 Startup 코드에 구현된 함수이다.

MMU_EnableICache()

- MMU I-cache enable

MMU_SetAsyncBusMode()

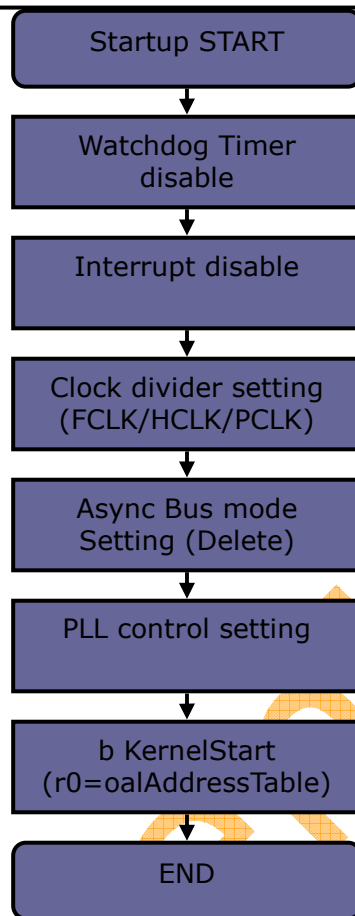
- MMU async bus mode setting

__RdPage512()

- Nand flash page(512B) read

6.7.3. Src\Kernel\Oal\startup.s

다음 그림은 kernel의 startup 코드 실행 순서를 나타내며, KernelStart() 함수를 호출함으로 커널 수행한다.



[Fig - 15] Kernel startup code flow chart

C-level 함수 호출을 위해 Kernel startup 코드에 구현된 함수이다.

OALCPUPowerOff()

- SLEEPDATA macro store and power off

Awake_address()

- Called by Step-stone loader startup.s
- Restore the SLEEPDATA macro

7. Debugging and Testing

The following table describes the debugging and testing tools available through Windows Mobile 5.0.

Debugging and testing tool	Description
System Debugging	The kernel debugger is a full-featured debugging system that controls the behavior of an operating system (OS) image. It stops all hardware interrupts, halts the entire OS, and



WinCE5.0 eBoot and Kernel Porting Guide

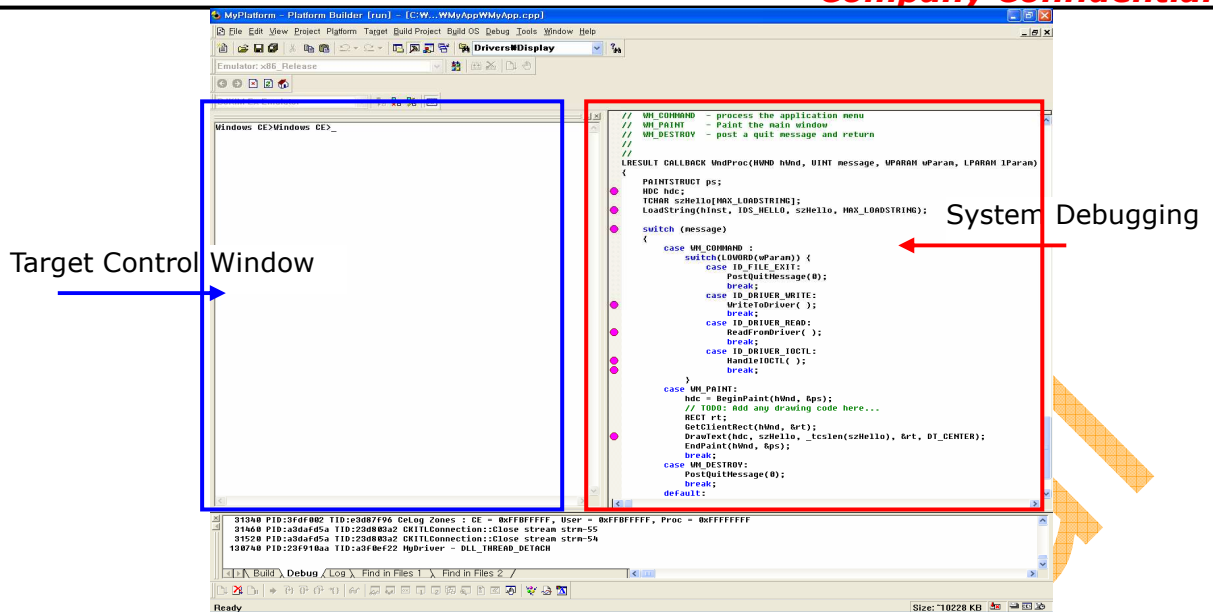
Version 0.6

Company Confidential

	provides full-featured debugging support.
Target Control Window	The Target Control dialog box provides useful command-line debugging options. (Can execute shell.exe commands.)
CeLog Event Tracking	The CeLog event tracking subsystem provides powerful and highly customizable event tracking. (CeLogFlush.exe creates a file named Celog.clg)
Hardware-assisted Debugging	Hardware-assisted debugging allows you to debug even when the target does not yet have a working bootloader or operating system.
Emulator	The Emulator simulates the behavior of a Windows CE-based target device on your development workstation.
Remote Tools	Remote Tools allow you to use a development workstation to remotely perform a variety of programming tasks on a Microsoft® Windows® CE-based target device.
Core Connectivity	The Core Connectivity infrastructure manages the communications between a desktop computer and a Windows CE-based target device. Core Connectivity allows you to download an application or run-time image to a target device, and to connect to that device for development and debugging. (For download image and transport service)
Windows CE Test Kit	The Windows CE Test Kit (CETK) is a tool that you can use to test device drivers that you develop for the Windows Mobile operating system (OS).

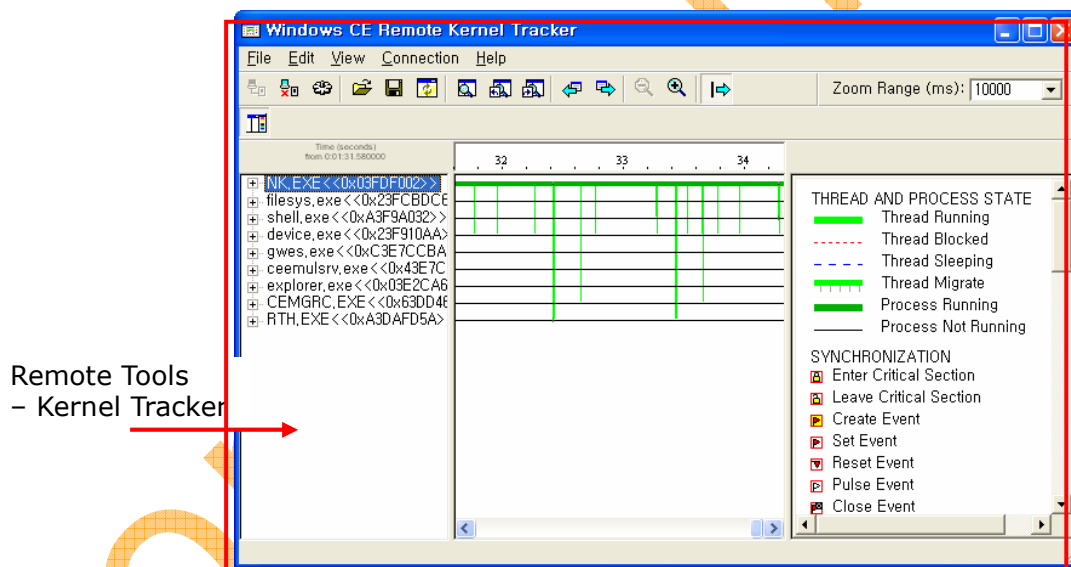
[Table - 6] Debugging Resource Overview

1) System Debugging and Target Control Window

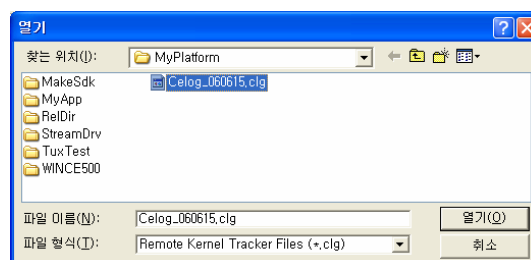


[Fig - 16] Platform Builder Execution Window

2) CeLog Event Tracking and Remote Tools



[Fig - 17] Remote Kernel Tracker Window



(When .cls file is loading in the Remote Kernel Tracker)

[Fig - 18] CeLog file Load Figure

7.1. Boot Loader

- 1) Test boot loader communications. (Serial or Ethernet connection etc.)
- 2) Verify that the boot loader can use Platform Builder to download a .bin file
Refer to [Verifying .bin File Download](#)
- 3) To verify the RAM or flash memory used by the run-time image before the run-time image is downloaded completely, add support for memory verification.
Refer to [Adding Support for Memory Verification](#)

7.2. Kernel

The Hardware-assisted Debugging method is debugging method from kernel development process. Others are the method when kernel operation is normal.

1) Hardware-assisted Debugging

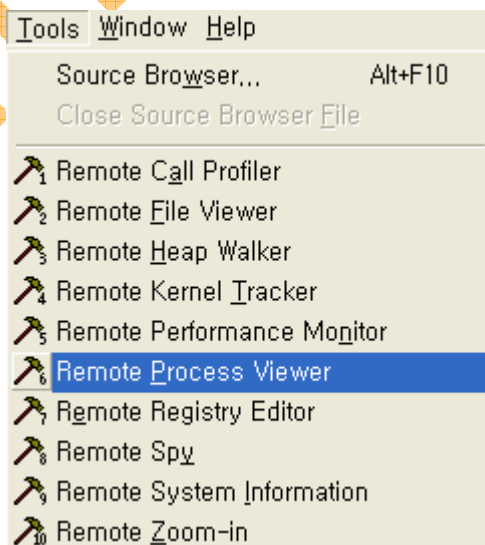
Third-party vendors provide the required hardware and software, which includes, at a minimum, a probe driver, and a probe. ex) Trace32, Jtag-Z, etc

2) System Debugging

can use the kernel debugger in the IDE to debug a run-time image on a target device.

3) Remote Tools

Ethernet or serial connection is valid.



[Fig - 19] Remote Tools in the Platform Builder

Remote tool	Description
-------------	-------------



WinCE5.0 eBoot and Kernel Porting Guide

Version 0.6

Company Confidential

Remote Call Profiler	Provides profiling and analysis tools within a graphical user interface (GUI) that enable you to identify algorithmic bottlenecks in your applications.
Remote File Viewer	Displays a hierarchical view of the file system on a target device.
Remote Heap Walker	Displays information about heap identifiers and flags for processes that are running on a target device.
Remote Kernel Tracker	Provides a visual representation of OS and application events occurring on a target device.
Remote Performance Monitor	Measures the performance of a target device.
Remote Process Viewer	Displays a list of processes and threads running on a target device.
Remote Registry Editor	Displays the registry for a target device and enables you to manage the registry.
Remote Spy	Displays a list of windows opened on a target device and messages generated by those windows.
Remote System Information	Displays system settings and properties for a target device.
Remote Zoom-in	Displays an image from the screen of a target device.

[Table - 7] Remote Tools in the Platform Builder

4) Emulator

APPENDIX

A. Reference (MSDN Site)

- 1) [WinCE5.0 BSP, Boot Loader, and Kernel Starup Sequence](#)
- 2) [How to Develop a Boot Loader](#)
- 3) [How to Develop an OAL\(OEM Adaptation Layer\)](#)
- 4) [Debugging and Testing](#)

B. Important Functions and Files

- 1) Boot Loader
- 2) Kernel

C. OAL Layer Functions

1) Required OAL Functions

1. CacheErrorHandler (MIPS platform specific function)
2. FlushDCache (Use the OEMCacheRangeFlush() instead)
3. FlushICache (Use the OEMCacheRangeFlush() instead)
4. InitClock (Use InitTimer()) - 수정
 - * Refer to PLATFORM\COMMON\SRC\ARM\SMT\SMT926A\TIMER\timer.c
 - * Refer to PLATFORM\SkyWalker50\Src\Kernel\oal\power.c
 - * OEMInit()에서 호출, InitClock() = OALTimerInit()
5. OEMARMCacheMode - 그대로 사용
 - * Refer to PLATFORM\COMMON\SRC\ARM\ARM926\CACHE\cachemode.s
6. OEMCacheRangeFlush - 그대로 사용
 - * Refer to PLATFORM\COMMON\SRC\ARM\ARM926\CACHE\flush.c
7. OEMClearDebugCommError - Don't care
 - * Just Refer to X86 (PLATFORM\...\X86\COMMON\OTHER\debug.c)
8. OEMDataAbortHandler - 일단 그대로 사용 (oal_arm920t.lilb)
 - * Refer to PLATFORM\COMMON\SRC\ARM\ARM920T\ABORT\oemabort.s
 - * Refer to private\winceos\coreos\NK\KERNEL\ARM\armtrap.s
9. OEMFlushCache - 구현 안되어 있음 (Don't care)
10. OEMGetExtensionDRAM - pNKEnumExtensionDRAM이용 (추후)
 - * Refer to PLATFORM\COMMON\SRC\COMMON\OTHER\memory.c)
11. OEMGetRealTime - 추후 (RTC 필요)
 - * Refer to PLATFORM\COMMON\SRC\ARM\SMT\SMT926A\Rtc\rtc.c

-
12. OEMIdle (OALCPUIdle()) 호출 - 그대로 사용
- * Refer to PLATFORM\COMMON\SRC\COMMON\TIMER\IDLE\idle.c
 - * Refer to PLATFORM\COMMON\SRC\ARM\SMT\SMT926A\Timer\timer.c
 - OALCPUIdle() : 수정
 - OALTimerCountsSinceSysTick() : 수정
- Refer to PLATFORM\COMMON\...\SMT926A\TIMER\timer.c
13. OEMInit - 일단 그대로 사용
- * Refer to PLATFORM\SkyWalker50\Src\Kernel\Oal\init.c
14. OEMInterruptDisable - 그대로 사용
- (OALIntrDisableIrqs() 호출, OALIntrTranslateSysIntr())
- OALIntrDisableIrqs() : 수정
 - * Refer to PLATFORM\COMMON\SRC\COMMON\INTR\COMMON\oem.c
 - * Refer to PLATFORM\COMMON\SRC\COMMON\INTR\BASE\map.c
 - * Refer to PLATFORM\COMMON\SRC\ARM\SMT\SMT926A\Intr\intr.c
15. OEMInterruptDone - 그대로 사용
- (OALIntrDoneIrqs() 호출, OALIntrTranslateSysIntr())
- OALIntrDoneIrqs() : 수정
 - * Refer to PLATFORM\COMMON\SRC\COMMON\INTR\COMMON\oem.c
 - * Refer to PLATFORM\COMMON\SRC\COMMON\INTR\BASE\map.c
 - * Refer to PLATFORM\COMMON\SRC\ARM\SMT\SMT926A\Intr\intr.c
16. OEMInterruptEnable - 그대로 사용
- (OALIntrEnableIrqs() 호출, OALIntrTranslateSysIntr())
- OALIntrEnableIrqs() : 수정
 - * Refer to PLATFORM\COMMON\SRC\COMMON\INTR\COMMON\oem.c
 - * Refer to PLATFORM\COMMON\SRC\COMMON\INTR\BASE\map.c
 - * Refer to PLATFORM\COMMON\SRC\ARM\SMT\SMT926A\Intr\intr.c
17. OEMInterruptHandler - 수정
- * Refer to PLATFORM\COMMON\SRC\ARM\SMT\SMT926A\Intr\intr.c
18. OEMInterruptHandlerFIQ - 추후
- * Refer to PLATFORM\COMMON\SRC\ARM\SMT\SMT926A\Intr\fiq.c
19. OEMIoControl - 그대로 사용
- * Refer to PLATFORM\COMMON\SRC\COMMON\IOCTL\ioctl.c
20. OEMNMI (SH4 platform specific)
21. OEMNMIIHandler (SH4 platform specific)
22. OEMPowerOff - 추후
- * Refer to PLATFORM\COMMON\SRC\ARM\SMT\SMT926A\Power\off.c
23. OEMSetAlarmTime - 추후
-

* Refer to PLATFORM\COMMON\SRC\ARM\SMT\SMT926A\Rtc\rtc.c

24. OEMSetRealTime - 추후

* Refer to PLATFORM\COMMON\SRC\ARM\SMT\SMT926A\Rtc\rtc.c

25. pReadRegistryFromOEM - 그대로 사용

* Refer to PLATFORM\COMMON\SRC\COMMON\PERREG\perreg.c

26. SC_GetTickCount - 그대로 사용

(OALTimerCountsSinceSysTick() 호출 - 수정)

* Refer to PLATFORM\COMMON\SRC\COMMON\TIMER\COMMON\counts.c

* Refer to PLATFORM\COMMON\SRC\ARM\SMT\SMT926A\Timer\timer.c

27. TLBClear (Use the OEMCacheRangeFlush() instead)

2) Optional OAL Functions

1. CertifyModule - Certification
2. CertifyModuleFinal - Certification
3. CertifyModuleInit - Certification
4. CreateInstance - Installable ISR
5. DestroyInstance - Installable ISR
6. FreeIntChainHandler - Installable ISR
7. IOControl - ISR & IST Comm
8. KLibAllocShareMem - ISR & IST Comm
9. KLibFreeShareMem - ISR & IST Comm
10. LoadIntChainHandler - Installable ISR
11. IpNKHaltSystem - Halt system
12. NKCallIntChain - Interrupt handler : **Don't care**

(Called OEMInterruptHandler() in the intr.c)

* Refer to private\winceos\coreos\NK\KERNEL\kdriver.c

* Refer to PLATFORM\COMMON\SRC\ARM\SMT\SMT926A\Intr\intr.c

13. NKCreateStaticMapping : **Don't care**

* Refer to private\winceos\coreos\NK\KERNEL\ARM\mdarm.c

14. NKForceCleanBoot : **Don't care**

* Refer to private\winceos\coreos\NK\KERNEL\loader.c

15. OEMCertifyModule - Certification
16. OEMCertifyModuleInit - Certification
17. OEMClearIntSources - Interrupt handler??
18. OEMEnumExtensionDRAM - Extension DRAM (OEMGetExtensionDRAM())
19. OEMEthISR - Don't use. See the KITLTRANSPORT structure.
20. OEMEthQueryClientInfo - Don't use. See the KITLTRANSPORT structure.
21. OEMFindFirstWakeSource - Interrupt handler



-
- * Just Refer to X86 (PLATFORM\...\X86\COMMON\POWER\power.c)
 - 22. OEMGetInterrupt - Interrupt handler
 - * Just Refer to X86 (PLATFORM\...\X86\COMMON\IO\route.c)
 - 23. OEMGetWakeupSource - Wakeup
 - * Just Refer to X86 (PLATFORM\...\X86\COMMON\POWER\power.c)
 - 24. OEMHandleVFPEException - ARM specific (Vector Floating Pointer H/W)
 - 25. OEMIndicateIntSource - Interrupt handler
 - * Just Refer to X86 (PLATFORM\...\X86\COMMON\POWER\power.c)
 - 26. OEMInitCoProRegisterSavedArea - Thread
 - 27. OEMIntrOccurs - Interrupt handler
 - 28. OEMIsRom - ROM address
 - 29. OEMKDIoControl - Kernel debug
 - 30. OEMPowerManageInit - Power management
 - 31. OEMRequestSysIntr - Interrupt handler
 - 32. OEMReschedule - Schedule
 - 33. OEMResetWakeupSource - Wakeup
 - * Just Refer to X86 (PLATFORM\...\X86\COMMON\POWER\power.c)
 - 34. OEMRestoreCoProcRegister - Thread
 - 35. OEMRestoreVFPCtrlRegs - ARM specific (Vector Floating Pointer H/W)
 - 36. OEMSaveCoProcRegister - Thread
 - 37. OEMSaveVFPCtrlRegs - ARM specific (Vector Floating Pointer H/W)
 - 38. OEMSetWakeupSource - Wakeup
 - * Just Refer to X86 (PLATFORM\...\X86\COMMON\POWER\power.c)
 - 39. OEMTranslateIrq - Interrupt handler
 - * Refer to OALIntrTranslateIrq()
 - * Refer to PLATFORM\COMMON\SRC\COMMON\INTR\BASE\map.c
 - 40. OEMTranslateSysIntr - Interrupt handler
 - * Refer to OALIntrTranslateSysIntr()
 - * Refer to PLATFORM\COMMON\SRC\COMMON\INTR\BASE\map.c
 - 41. OEMWriteDebugLED - LED : Don't care
 - 42. pfnNKDrWatsonClear - DrWatsonClear???
 - * Refer to private\winceos\coreos\NK\KERNEL\kdriver.c
 - 43. pfnNKDrWatsonFlush - DrWatsonFlush???
 - * Refer to private\winceos\coreos\NK\KERNEL\kdriver.c
 - 44. pfnNKDrWatsonRead - DrwatsonRead???
 - * Refer to private\winceos\coreos\NK\KERNEL\kdriver.c
 - 45. pfnNKDrWatsonWrite - DrWatsonWrite???
 - * Refer to private\winceos\coreos\NK\KERNEL\kdriver.c
-

-
- 46. pfnOEMRefreshWatchDog - Watchdog Timer function pointer
 - * Refer to PLATFORM\COMMON\...\ARM\SMT\SMT926A\Timer\watchdog.c
 - 47. pfnOEMIntrOccurs - Interrupt handler function pointer
 - 48. pfnOEMReschedule - OEMReschedule function pointer
 - 49. pfnOEMSetMemoryAttributes - Memory
 - * Just Refer to PLATFORM\COMMON\...\X86\COMMON\MEMORY\memory.c
 - 50. pNKEnumExtensionDRAM - RAM
 - 51. pNotifyForceCleanboot - Clean boot
 - 52. pOEMCalcFSPages - Memory
 - * Just Refer to private\..\KERNEL\loader.c
 - 53. pOEMIsProcessorFeaturePresent - IsProcessorFeaturePresent function pointer
 - 54. pOEMLoadInit - OEMCertifyModuleInit function pointer
 - 55. pOEMLoadModule - OEMCertifyModule function pointer
 - 56. pOEMUpdateRescheduleTime - Timer
 - * Refer to PLATFORM\COMMON\...\COMMON\TIMER\VARTICK\vartick.c
 - 57. pQueryPerformanceCounter - Timer
 - * Refer to PLATFORM\COMMON\...\COMMON\TIMER\COMMON\counts.c
 - * Refer to PLATFORM\COMMON\...\ARM\SMT\SMT926A\Timer\timer.c
 - 58. pQueryPerformanceFrequency - Timer
 - * Refer to PLATFORM\COMMON\...\COMMON\TIMER\COMMON\counts.c
 - * Refer to PLATFORM\COMMON\...\ARM\SMT\SMT926A\Timer\timer.c
 - 59. pWriteDebugLED - LED
 - 60. pWriteRegistryToOEM - Registry
 - * Refer to PLATFORM\COMMON\...\COMMON\PERREG\perreg.c
 - 61. ReadRegistryFromOEM - Registry
 - * Refer to PLATFORM\COMMON\...\COMMON\PERREG\perreg.c
 - 62. WriteRegistryToOEM - Registry
 - * Refer to PLATFORM\COMMON\...\COMMON\PERREG\perreg.c

3) Required ILTiming Functions

- 1. PerfCountFreq - **Need to call trace**
 - * Refer to PLATFORM\COMMON\SRC\ARM\INTEL\PXA250\TIMER\vartick.c
 - * Refer to PLATFORM\COMMON\SRC\ARM\INTEL\PXA250\TIMER\iltiming.c
- 2. PerfCountSinceTick
 - * Refer to PLATFORM\COMMON\SRC\COMMON\ILT\COMMON\ilt.c
 - OALTimerCountsSinceSysTick() 호출 : 수정 (Smt926a\timer\timer.c)
 - * Refer to PLATFORM\COMMON\SRC\ARM\INTEL\PXA250\TIMER\vartick.c

4) Required KITL Functions

1. KitlInit

KITL subsystem 초기화하기 위해 OEM에 의해 호출 된다.

- OEMKitlInit() : **그대로 사용**

* Refer to PLATFORM\COMMON\SRC\COMMON\KITL\kitl.c

2. KitlSendRawData

* Refer to private\winceos\coreos\NK\KITL\edbgprot.c

3. KitlSetTimerCallback

* Refer to private\winceos\coreos\NK\KITL\timer.c

4. KitlStopTimerCallback

* Refer to private\winceos\coreos\NK\KITL\timer.c

5. OEMEthDisableInts - Don't use as of 4.2. See the KITLTRANSPORT structure.

6. OEMEthEnableInts - Don't use as of 4.2. See the KITLTRANSPORT structure.

7. OEMEthInit - Don't use as of 4.2. See the KITLTRANSPORT structure.

8. OEMKitlGetSecs

현재 시간의 초 단위를 리턴한다.

9. OEMKitlInit

* Refer to PLATFORM\COMMON\SRC\COMMON\KITL\kitl.c

* COMMON\kitl.c 사용 시 OALKitlEthInit / OALKitlSerialInit 함수 구현되어야

* **OALKitlStart** 사용 (SkyWalker)

-> Refer to Oal/kitl.c and init.c

- InitKitlSerialArgs()

- InitKitlEtherArgs() : **추후 (포팅 필요)**

- InitKitlUSBSerialArgs() : **추후 (포팅 필요)**

10. TransportDecode

OEMKitlInit 시 **KITLTRANSPORT** 구조체 멤버 pfnDecode에 할당

11. TransportEnableInt

OEMKitlInit 시 KITLTRANSPORT 구조체 멤버 pfnDecode에 할당

12. TransportGetDevCfg

OEMKitlInit 시 KITLTRANSPORT 구조체 멤버 pfnDecode에 할당

13. TransportRecv

OEMKitlInit 시 KITLTRANSPORT 구조체 멤버 pfnDecode에 할당

14. TransportSend

OEMKitlInit 시 KITLTRANSPORT 구조체 멤버 pfnDecode에 할당

15. TransportSetHostCfg

OEMKitlInit 시 KITLTRANSPORT 구조체 멤버 pfnDecode에 할당

5) Optional KITL Functions**1. CallEdbgDeregisterClient**

- * Refer to private\winceos\coreos\FSD\RELFSD\cefs.c
- * Refer to private\winceos\coreos\NK\KITL\ethdbg.c

2. CallEdbgRecv

- * Refer to private\winceos\coreos\FSD\RELFSD\cefs.c
- * Refer to private\winceos\coreos\NK\KITL\ethdbg.c

3. CallEdbgRegisterClient

- * Refer to private\winceos\coreos\FSD\RELFSD\cefs.c
- * Refer to private\winceos\coreos\NK\KITL\ethdbg.c

4. CallEdbgSend

- * Refer to private\winceos\coreos\FSD\RELFSD\cefs.c
- * Refer to private\winceos\coreos\NK\KITL\ethdbg.c

5. CallEdbgSetDebug

- * Refer to private\winceos\coreos\FSD\RELFSD\cefs.c
- * Refer to private\winceos\coreos\NK\KITL\ethdbg.c

6. KITLOutputDebugString

- * Refer to private\winceos\coreos\NK\KITL\debug.c

6) Required Profiling Functions**1. OEMProfileTimerDisable**

- 그대로 사용

- * Refer to PLATFORM\COMMON\SRC\ARM\SMT\SMT926A\Profiler\profiler.c

2. OEMProfileTimerEnable

- 그대로 사용

- * Refer to PLATFORM\COMMON\SRC\ARM\SMT\SMT926A\Profiler\profiler.c

7) Other OAL Functions

Further work

D. OAL Global Variables

1) Required OAL Variables**1. CurMSec**

- : Indicates the number of milliseconds since boot
- * Refer to private\winceos\coreos\NK\KERNEL\ARM\mdarm.c

2. dwReschedTime

- : Stores the time, in ticks, when a reschedule event must be triggered by the OEM
- * Refer to private\winceos\coreos\NK\KERNEL\schedule.c

3. OEMAddressTable

* Refer to PLATFORM\SkyWalker50\Src\Inc\oemaddrtab_cfg.inc

4. bProfileTimerRunning

: Indicates if the Monte Carlo profiling timer is running or not.

* Refer to private\winceos\coreos\NK\KERNEL\profiler.c

2) Optional OAL Variables

1. cbNKCoProcRegSize

This global variable describes the size of the memory allocation, in bytes, needed for the OEM to save or restore coprocessor registers.

2. CEProcessorType

This global variable is used by the kernel to assign the microprocessor type to the **dwProcessorType** member of [SYSTEM_INFO](#).

How do we implement the PROCESSOR_ARM926???
(PROCESSOR_ARM920 만 있음)

3. curidlehigh

This global variable contains the high **DWORD** of the 64-bit idle time and is used to support [GetIdleTime](#).

4. curidlelow

This global variable contains the low **DWORD** of the 64-bit idle time and is used to support [GetIdleTime](#)

5. DiffMSec - Obsolete

6. dwCeLogLargeBuf

This global variable indicates the number of milliseconds since the last time the scheduler examined the sleep queue

7. dwCeLogSmallBuf

This global variable is used by the OEM to adjust the size of the secondary logging buffer. This buffer is used to log interrupt events and must not be larger than 50% of the size of *dwCeLogLargeBuf*.

8. dwCeLogTLBMiss

This global variable specifies how many translation look-aside buffer (TLB) misses have occurred for the last thread that was running.

9. deDefalutThreadQuantum

This variable sets the default thread quantum of all threads.

10. dwNKAlarmResolutionMSec

This variable allows the OEM to specify the resolution of the kernel alarm



timer to 1 second

11. dwNKCoProcEnableBits - MIPS specific

12. dwNKDrWatsonSize

This variable specifies the amount of memory for the Windows CE Error Reporting dump area

13. dwNKMaxPrioNoScav

This variable defines the highest priority for a thread that should not incur stack space recovery

14. dwNKWatchDogThreadPriority

This variable specifies the kernel watchdog thread

15. dwOEMCleanPages

This variable is set to signal to the kernel how much memory should be set to zero to enable the device to boot.

Any memory not zeroed at boot is zeroed in a separate thread

16. dwOEMPTPoolSize - Obsolete. Don't use (as of 4.2)

17. dwOEMWatchDogPeriod

This variable specifies the watchdog period, in milliseconds, where the hardware watchdog must be refreshed before system reset

18. dwPartialDiffMSec - Obsolete

19. dwPreempt - Obsolete

20. dwSleepMin - Obsolete

21. fNKSaveCoProcReg

This global variable is used as a flag to tell the kernel when the save/restore of coprocessor registers is performed

22. fUseIterations - Obsolete

23. idleconv

This global variable is a translation constant that specifies the number of units per millisecond and is used to support [GetIdleTime](#)

24. MainMemoryEndAddress

This global variable specifies the next available address following the first available contiguous block of memory

25. pfnOEMIsRom

This variable is a pointer to an [OEMIsRom](#) function

26. OEMMemoryMap - Obsolete (Use OEMAddressTable instead)

27. OEMTLBSize

This variable is declared by the OEM and defines how many TLB entries are supported